

Basic Optimization

Gautam Kamath

Optimization methods

- Initialize w_0
- For $t = 1, 2, \dots$
 - Choose search direction v_t and step size η_t
 - $w_t = w_{t-1} - \eta_t v_t$
- Which direction v_t ?
- How far η_t ?

Which direction v_t ?

- Suppose we have some function f , a current parameter vector w
 - $f(w)$
- How does f behave when we shift w by some small vector Δ ?
 - Taylor expansion: $f(w + \Delta) \approx f(w) + \nabla_w f(w)^T \Delta + \mathcal{O}(\|\Delta\|^2)$
- Which Δ minimizes f ? (In a small local neighborhood)
 - $\Delta \propto -\nabla f(w)$
- Take a step in the negative gradient of the function f at w

Gradient descent

- Goal: minimize $f(w) = \mathbf{E}[\ell_w(x, y)]$
- Recall: in ERM, we minimize $\tilde{f}(w) = \frac{1}{n} \sum_i \ell_w(x_i, y_i)$ as a proxy
- $w_t \leftarrow w_{t-1} + \Delta_{t-1} = w_{t-1} - \eta_t \nabla_w \frac{1}{n} \sum_{i=1}^n \ell_{w_{t-1}}(x_i, y_i)$
 - Repeat for T steps
 - Gradient descent
 - How long does it take?
- For a set $B \subseteq [n]$, $w_t \leftarrow w_{t-1} - \eta_t \nabla_w \frac{1}{|B|} \sum_{i \in B} \ell_{w_{t-1}}(x_i, y_i)$
 - (Mini-batch) stochastic gradient descent
 - Lower cost per step, noisier (unbiased?) estimate of the full gradient

Step sizes η_t

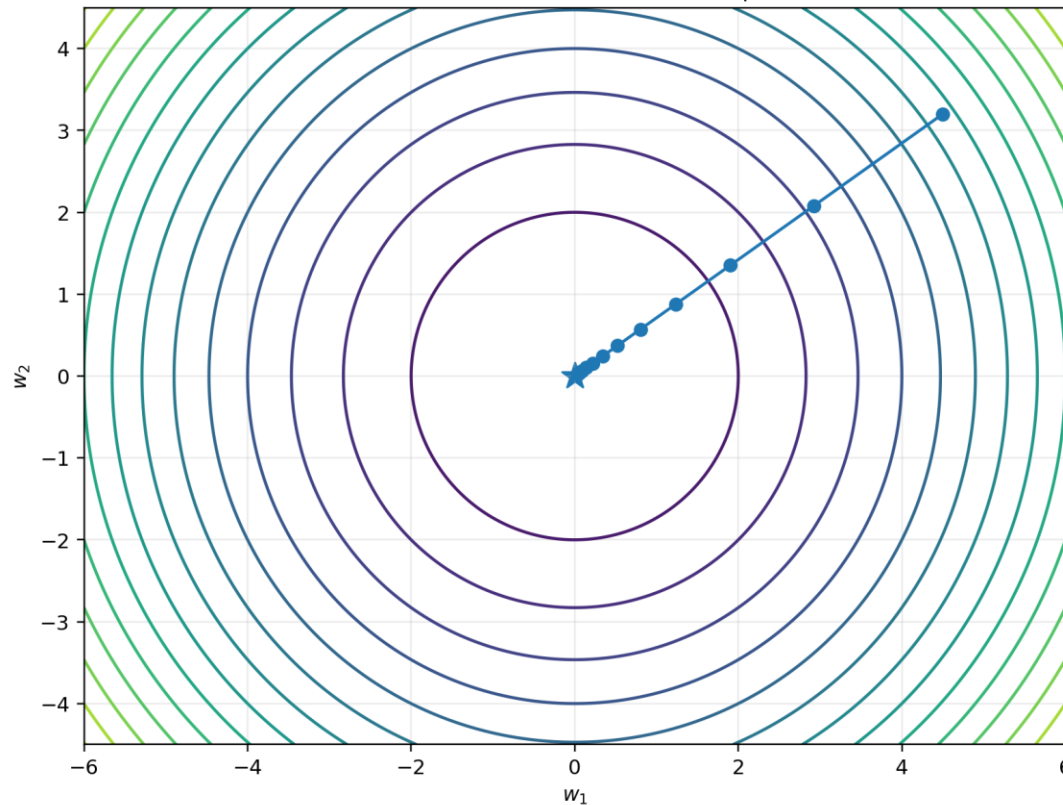
- (Draw picture on quadratic, step size too small, big, and just right)
- Schedules for step sizes
 - Constant
 - Decaying (e.g., $1/\sqrt{t}$)
 - Adaptive

Conditioning

Condition number κ : ratio of largest and smallest eigenvalues

$$f(w) = \frac{1}{2}(w_1^2 + w_2^2)$$

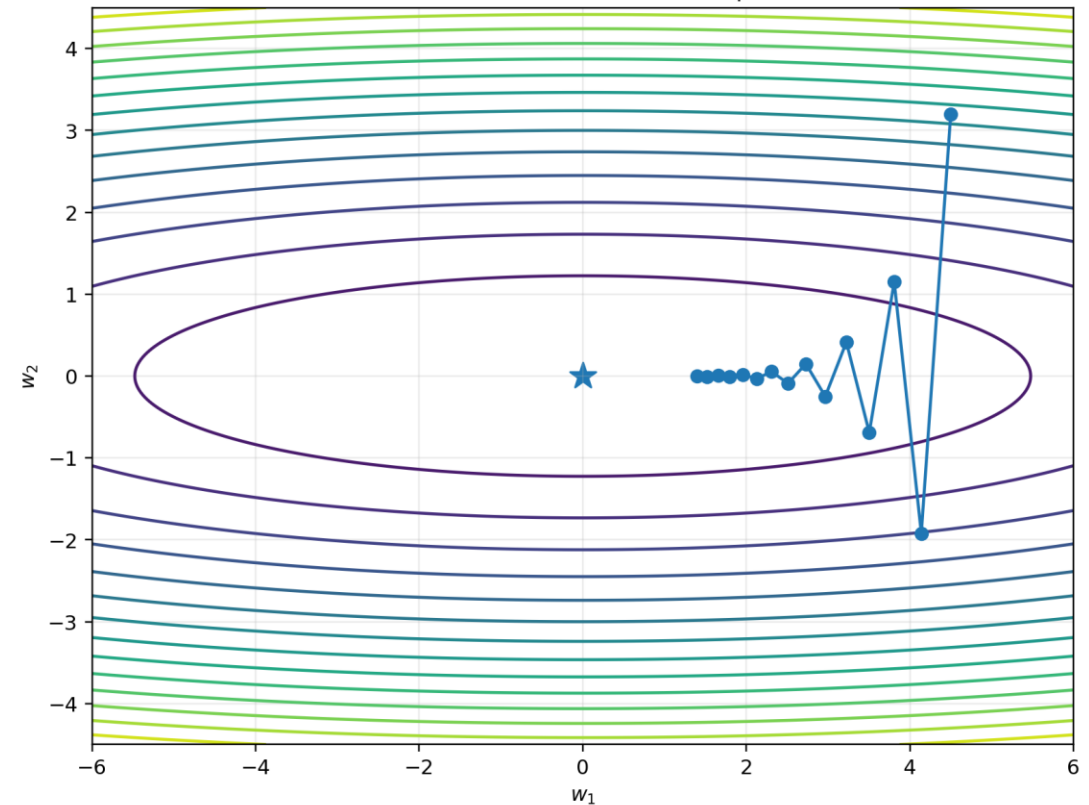
Gradient descent: well-conditioned quadratic



$$H(w) = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \kappa = 1 \quad -\nabla f(w) = (-w_1, -w_2)$$

$$f(w) = \frac{1}{2}(w_1^2 + 20w_2^2)$$

Gradient descent: ill-conditioned quadratic



$$H(w) = \begin{pmatrix} 1 & 0 \\ 0 & 20 \end{pmatrix}, \kappa = 20 \quad -\nabla f(w) = (-w_1, -20w_2)$$

Newton's method

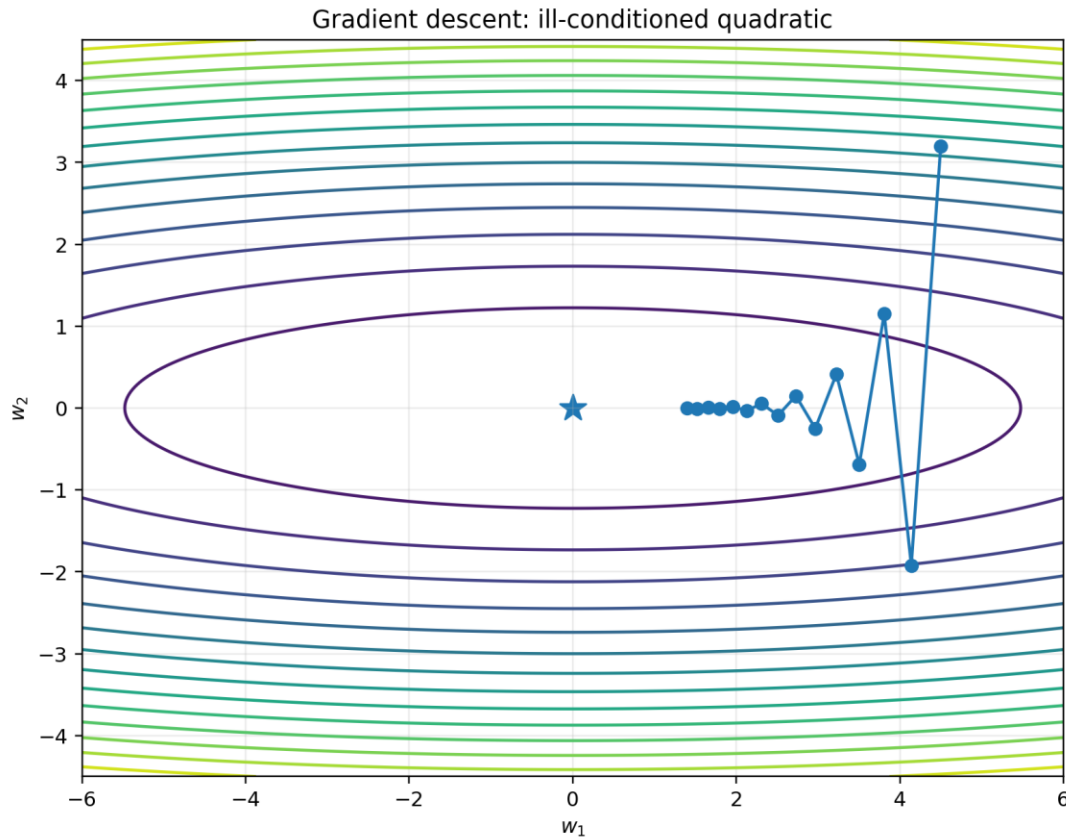
- How does f behave when we shift w by some small vector Δ ?
- Second-order Taylor expansion (versus first-order before)
- $f(w + \Delta) \approx f(w) + \nabla_w f(w)^T \Delta + \frac{1}{2} \Delta^T \nabla_w^2 f(w) \Delta + \dots$
- Which Δ minimizes this? Set derivative = 0.
- $\nabla_{\Delta} \left(f(w) + \nabla_w f(w)^T \Delta + \frac{1}{2} \Delta^T \nabla_w^2 f(w) \Delta \right) = 0$
- $\nabla_w f(w) + \nabla_w^2 f(w) \Delta = 0$
- $\Delta = -\left(\nabla_w^2 f(w) \right)^{-1} \nabla_w f(w)$
- Δ is the negative gradient, but scaled by the *inverse Hessian*
- *Exactly* minimizes the quadratic approximation (cf. GD)

Newton's method

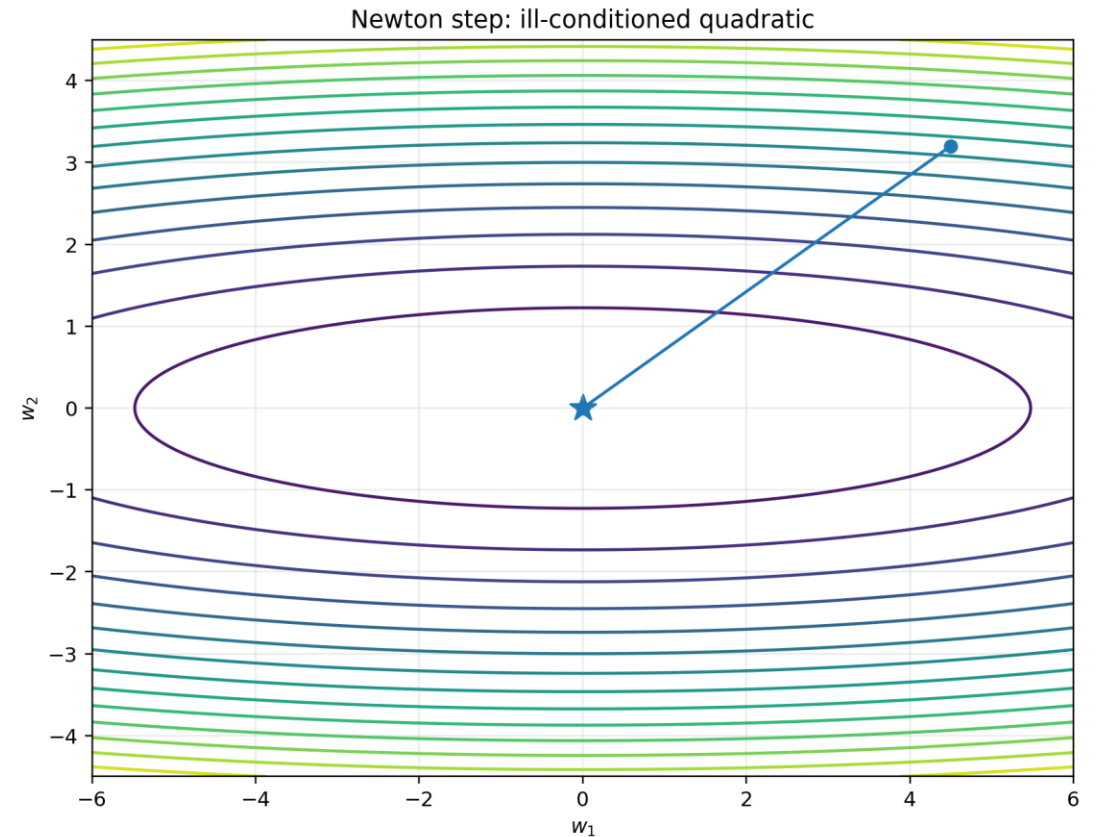
- $v_t = \left(\frac{1}{n} \sum_{i=1}^n \nabla_w^2 \ell_{w_{t-1}}(x_i, y_i) \right)^{-1} \left(\frac{1}{n} \sum_{i=1}^n \nabla_w \ell_{w_{t-1}}(x_i, y_i) \right)$
- Fewer steps to converge
- But more time and memory required for each step
 - How much?
- Quasi-Newton methods try to avoid this

Newton on ill-conditioned

$$f(w) = \frac{1}{2}(w_1^2 + 20w_2^2)$$



$$-\nabla f(w) = (-w_1, -20w_2)$$



$$-H^{-1}\nabla f(w) = \begin{pmatrix} 1 & 0 \\ 0 & 20 \end{pmatrix}^{-1} (-w_1, -20w_2) = (-w_1, -w_2)$$